FC7xxx DMA Integration Manual

Rev.0.4

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/11/22	Updated for MCAL V0.4.0

Table of Contents

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the DMA module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides peripherals PCC assignment for Dma module (include Dma and Dma_mux).
- Det: The configuration item "DmaDevErrorDetect" depends on Det.
- Common: This module is the basic module which used to choose the chip.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).

Module initialization dependency

- Mcu: To use the Dma module, the user needs to initialize the MCU module first.

2.2 Files Required for Compile

Dma module files:

- _MCAL/Src/DMA/Src/CDD_Dma.c
- _MCAL/Src/DMA/Src/Dma_HWA.c
- _MCAL/Src/DMA/Src/Dma_Isr.c
- _MCAL/Src/DMA/Src/Dma_LLD.c
- _MCAL/Src/DMA/include/CDD_Dma.h
- _MCAL/Src/DMA/include/Dma_HWA.h
- _MCAL/Src/DMA/include/Dma_LLD.h
- _MCAL/Src/DMA/include/Dma_Memmap.h
- _MCAL/Src/DMA/include/Dma_Reg.h
- _MCAL/Src/DMA/include/Dma_Types.h
- _MCAL/Src/DMA/include/DmaMux_HWA.h
- _MCAL/Src/DMA/include/DmaMux_Reg.h

- _MCAL_generate/src/CDD_DMA_Cfg.c
- _MCAL_generate/include/ CDD_DMA_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Dma.h

2.3 Add Plug-ins

DMA module plug-ins are developed for EB tresos Studio, so, to use DMA plug-ins on the EB tresos Studio, the user needs to add the DMA module to the EB plug-ins folder first.

1) Copy the DMA module(_MCAL/EB_Plugins/eclipse/plugins/DMA) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
2) Set the DMA module output location folder for the generated source file and header files.
3) Use EB tresos to configure the DMA module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
DMA_START_SEC_CONFIG_DATA_8 DMA_STOP_SEC_CONFIG_DATA_8 DMA_START_SEC_CONFIG_DATA_16 DMA_STOP_SEC_CONFIG_DATA_16 DMA_START_SEC_CONFIG_DATA_32 DMA_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (rodata).
DMA_START_SEC_CONFIG_DATA_UNSPECIFIED DMA_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data (rodata).
DMA_START_SEC_CONST_BOOLEAN DMA_STOP_SEC_CONST_BOOLEAN DMA_START_SEC_CONST_8 DMA_STOP_SEC_CONST_8 DMA_START_SEC_CONST_16 DMA_STOP_SEC_CONST_16 DMA_START_SEC_CONST_32 DMA_STOP_SEC_CONST_32 DMA_START_SEC_CONST_ UNSPECIFIED DMA_STOP_SEC_CONST_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
DMA_START_SEC_CODE DMA_STOP_SEC_CODE DMA_START_SEC_ RAMCODE DMA_STOP_SEC_ RAMCODE DMA_START_SEC_CODE_AC DMA_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
DMA_START_SEC_VAR_NO_INIT_BOOLEAN DMA_STOP_SEC_VAR_NO_INIT_BOOLEAN DMA_START_SEC_VAR_NO_INIT_8 DMA_STOP_SEC_VAR_NO_INIT_8 DMA_START_SEC_VAR_NO_INIT_16 DMA_STOP_SEC_VAR_NO_INIT_16 DMA_START_SEC_VAR_NO_INIT_32 DMA_STOP_SEC_VAR_NO_INIT_32 DMA_START_SEC_VAR_NO_INIT_ UNSPECIFIED DMA_STOP_SEC_VAR_NO_INIT_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
DMA_START_SEC_VAR_INIT_BOOLEAN DMA_STOP_SEC_VAR_INIT_BOOLEAN DMA_START_SEC_VAR_INIT_8 DMA_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
DMA_START_SEC_VAR _INIT_16 DMA_STOP_SEC_VAR_INIT_16 DMA_START_SEC_VAR_INIT_32 DMA_STOP_SEC_VAR_INIT_32 DMA_START_SEC_VAR_INIT_ UNSPECIFIED DMA_STOP_SEC_VAR_INIT_ UNSPECIFIED		startup code (data).
DMA_START_SEC_VAR_NO_INIT_SHARE_MEMORY DMA_STOP_SEC_VAR_NO_INIT_SHARE_MEMORY DMA_START_SEC_VAR_INIT_SHARE_MEMORY DMA_STOP_SEC_VAR_INIT_SHARE_MEMORY	Variables	These are all the sections used for variables which have to be defined in SRAM for multicore interaction. These variables are initialized by startup code (data).

Chapter 4 Exclusive Area

DMA module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the DMA driver:

- CDD_Dma.c
  - SchM_Enter_Dma_DMA_EXCLUSIVE_AREA_00 is used in Dma_DeInit.
- Dma_LLD.c
  - SchM_Enter_Dma_DMA_EXCLUSIVE_AREA_01 is used in Dma_LLD_Init.
  - SchM_Enter_Dma_DMA_EXCLUSIVE_AREA_02 is used in Dma_LLD_CheckCircularBuffer.
  - SchM_Enter_Dma_DMA_EXCLUSIVE_AREA_03 is used in Dma_LLD_SetOuterLinkChannel.
  - SchM_Enter_Dma_DMA_EXCLUSIVE_AREA_04 is used in Dma_LLD_Init.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
DMA 0 / DMA 1	DMA channel 0 transfer complete	0
DMA 0 / DMA 1	DMA channel 1 transfer complete	1
DMA 0 / DMA 1	DMA channel 2 transfer complete	2
DMA 0 / DMA 1	DMA channel 3 transfer complete	3
DMA 0 / DMA 1	DMA channel 4 transfer complete	4
DMA 0 / DMA 1	DMA channel 5 transfer complete	5
DMA 0 / DMA 1	DMA channel 6 transfer complete	6
DMA 0 / DMA 1	DMA channel 7 transfer complete	7
DMA 0 / DMA 1	DMA channel 8 transfer complete	8
DMA 0 / DMA 1	DMA channel 9 transfer complete	9
DMA 0 / DMA 1	DMA channel 10 transfer complete	10
DMA 0 / DMA 1	DMA channel 11 transfer complete	11
DMA 0 / DMA 1	DMA channel 12 transfer complete	12
DMA 0 / DMA 1	DMA channel 13 transfer complete	13
DMA 0 / DMA 1	DMA channel 14 transfer complete	14
DMA 0 / DMA 1	DMA channel 15 transfer complete	15
DMA 0 / DMA 1	DMA channel 16 transfer complete	16
DMA 0 / DMA 1	DMA channel 17 transfer complete	17
DMA 0 / DMA 1	DMA channel 18 transfer complete	18
DMA 0 / DMA 1	DMA channel 19 transfer complete	19
DMA 0 / DMA 1	DMA channel 20 transfer complete	20
DMA 0 / DMA 1	DMA channel 21 transfer complete	21
DMA 0 / DMA 1	DMA channel 22 transfer complete	22
DMA 0 / DMA 1	DMA channel 23 transfer complete	23
DMA 0 / DMA 1	DMA channel 24 transfer complete	24
DMA 0 / DMA 1	DMA channel 25 transfer complete	25
DMA 0 / DMA 1	DMA channel 26 transfer complete	26
DMA 0 / DMA 1	DMA channel 27 transfer complete	27
DMA 0 / DMA 1	DMA channel 28 transfer complete	28
DMA 0 / DMA 1	DMA channel 29 transfer complete	29
DMA 0 / DMA 1	DMA channel 30 transfer complete	30
DMA 0 / DMA 1	DMA channel 31 transfer complete	31
DMA 0 / DMA 1	DMA channel 32 transfer complete	32
DMA 0 / DMA 1	DMA error interrupt channels 0-63	33

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Dma_Init	DMA_E_ALREADY_INITIALIZED_U8 DMA_E_UNINIT_U8 DMA_E_INIT_FAILED_U8 DMA_E_PARTITION_MAPPING
Dma_DeInit	DMA_E_ALREADY_INITIALIZED_U8 DMA_E_UNINIT_U8 DMA_E_PARTITION_MAPPING
Dma_GetVersionInfo	DMA_E_PARAM_VINFO_U8
Dma_CancelTansfer	DMA_E_PARTITION_MAPPING
Dma_ErrorCancelTansfer	DMA_E_PARTITION_MAPPING
Dma_Halt	DMA_E_PARTITION_MAPPING
Dma_Resume	DMA_E_PARTITION_MAPPING
Dma_ConfigChannel	DMA_E_PARAM_VINFO_U8 DMA_E_INVALID_CHANNEL_U8
Dma_StartChannel	DMA_E_INVALID_CHANNEL_U8
Dma_SetInnerLinkChannel	DMA_E_INVALID_CHANNEL_U8
Dma_SetOuterLinkChannel	DMA_E_INVALID_CHANNEL_U8
Dma_SetChannelPriority	DMA_E_INVALID_CHANNEL_U8
Dma_SetChannelLoopOffsetAndNBYTES	DMA_E_INVALID_CHANNEL_U8 DMA_E_PARAM_VINFO_U8
Dma_CheckIfTransferCompleted	DMA_E_INVALID_CHANNEL_U8
Dma_CheckIfTransferActive	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgSlast	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgSaddr	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgSoff	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgDlast	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgDaddr	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgDoff	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgSModuloAndSize	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgDModuloAndSize	DMA_E_INVALID_CHANNEL_U8
Dma_EnableHwRequest	DMA_E_INVALID_CHANNEL_U8
Dma_DisableHwRequest	DMA_E_INVALID_CHANNEL_U8
Dma_SetCfgCompleteInterrupt	DMA_E_INVALID_CHANNEL_U8

6.2 Dem

N/A

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs to be called is "Dma_Init" and "Dma_ConfigChannel". When starting DMA handling, for hardware triggered channels, API "Dma_EnableHwRequest" should be used, and for software triggered channels, API "Dma_StartChannel" should be used.

7.2 Function Calls during Shutdown

The API needs to be called is "Dma_DeInit".

7.3 Function Calls during Wake-up

N/A

7.4 Function Calls during Runtime

When halting DMA handling, API "Dma_Halt" should be used. When resuming DMA handling, API "Dma_Resume" should be used. When canceling the remaining data transfer, API "Dma_CancelTansfer" should be used.

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

When using hardware triggering, it is important to pay attention to the configuration of the callout function interface. Most peripheral modules require this callout function to ensure the normal operation of other peripheral module state machines or other functions.

- Notification
  None
- Callback
  None
- Callout
  If the user enables the DMA complete interrupt or DMA error interrupt, and the callout value is not "NULL_PTR" or "NULL", an extern declaration will generate in CDD_DMA_Cfg.c. User can implement the notification in any file.

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:
  AUTOSAR_OS_NOT_USED need defined.

  In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example：

```
extern ISR(DMA0_Done_Isr);
void DMA0_IRQHandler (void)
{
DMA0_Done_Isr();
}
```

- If OS is used:
  Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

1) Configure the DMA module and generate configuration files (please refer to Building chapter for details).
2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
3) Coding with detailed channel configuration.
4) Map interrupt notification to their vector locations (please refer to chapter ISR for details) and enable the corresponding NVIC_ ISER.
5) Build the DMA module with dependent modules.